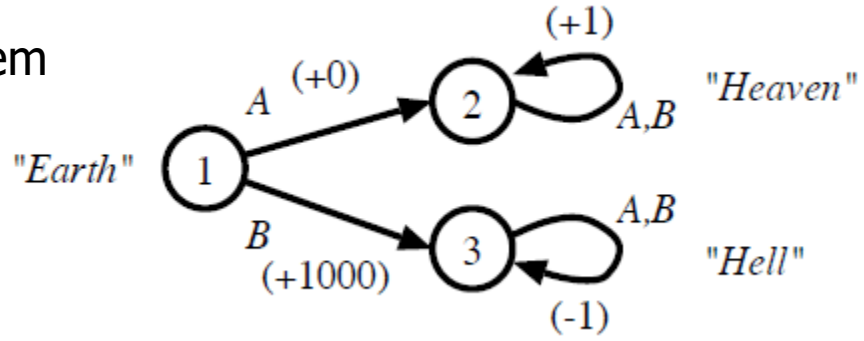
IE540
L13. Average Reward MDP, Finite Horizon MDP

KAIST ISE
Hayong Shin

Why discount? $(\gamma < 1)$

- Why discount? $(\gamma < 1)$ ← from Lecture 2
  - Financial reason
  - Human or animal agents are myopic indeed
  - Uncertainty about the future
  - For mathematical convenience
    - especially for continuing task
      - return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$ → ∞ if $\gamma = 1$

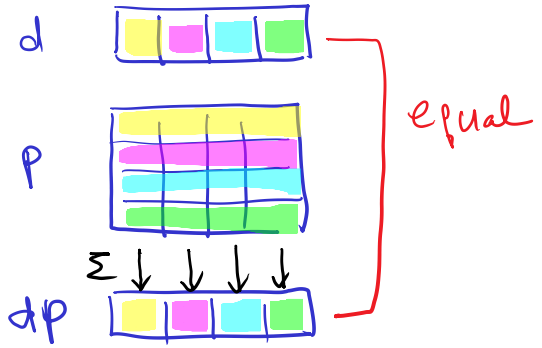
- Discounting may distort the problem
  - $\gamma = 0.99$
  - $Q(1, A) =$
  - $Q(1, B) =$



- Average reward setting (for undiscounted continuing task)
  - instead of discounted return $J_0(\pi) = \mathbb{E}_{\tau \sim p_0, \pi}[R(\tau)] = \mathbb{E}_{\tau \sim p_0, \pi}[\sum_{t=0}^{\infty} \gamma^t r_t]$
  - use average reward per step

Average reward setting (no discounting, $\gamma = 1$)

- Stationary distribution
  - $d^\pi(s) = \lim_{t\to\infty} p(S_t = s|\pi) = \sum_{s_0} p_0(s_0) \sum_{k=0}^{\infty} p(s_0 \to s, k, \pi)$
  - $d^\pi = d^\pi P^\pi$
    - $p^\pi_{s,s'} = \sum_{a\in\mathcal{A}} \pi(a|s) p(s'|s,a)$
    - $P^\pi = [p^\pi_{s,s'}]$
    - ➔ $\sum_{s\in\mathcal{S}} d^\pi(s) \sum_{a\in\mathcal{A}} \pi(a|s) p(s'|s,a) = d^\pi(s')$



- Average reward (per step)
  - $\bar{r}(\pi) = \lim_{h\to\infty} \frac{1}{h} \sum_{t=0}^{h-1} \mathbb{E}_{s_0\sim p_0,\, a_t\sim\pi}[r(s_t, a_t)]$: trajectory view
  - $\bar{r}(\pi) = J_{ER}(\pi) = \mathbb{E}_{s\sim d^\pi,\, a\sim\pi}[r(s,a)]$: stationary view
    - $\bar{r}(\pi) = \sum_{s\in\mathcal{S}} d^\pi(s) \sum_{a\in\mathcal{A}} \pi(a|s) r(s,a)$

Relative return, relative value function

In RLAI 2.0 book, 'differential' is used in place of 'relative'

cf. Discounted Return
$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$

- Relative return
  - relative reward $(R_t - \bar{r}(\pi))$
  - relative return $G_t = \sum_{k=0}^{\infty}(R_{t+k} - \bar{r}(\pi))$
    - excessive return than average
- relative value function
  - $V^{\pi}(s) = \mathbb{E}_{\pi}[G_t|S_t = s] = \mathbb{E}_{\pi}[\sum_{k=0}^{\infty}(R_{t+k} - \bar{r}(\pi))\,|S_t = s]$
  - $Q^{\pi}(s,a) = \mathbb{E}_{\pi}[G_t|S_t = s, A_t = a] = \mathbb{E}_{\pi}[\sum_{k=0}^{\infty}(R_{t+k} - \bar{r}(\pi))\,|S_t = s, A_t = a]$
- Bellman expectation equation (on relative value function)
  - $\bar{r}(\pi) + V^{\pi}(s) = \mathbb{E}_{a\sim\pi(.|s),(s',r)\sim p(s',r|s,a)}[r + V^{\pi}(s')]$
  - $\bar{r}(\pi) + Q^{\pi}(s,a) = \mathbb{E}_{(s',r)\sim p(s',r|s,a),a'\sim\pi(.|s')}[r + Q^{\pi}(s',a')]$
- Bellman optimality equation (on relative value function)
  - $\bar{r}^* + V^*(s) = \max_a \left\{\mathbb{E}_{(s',r)\sim p(s',r|s,a)}[r + V^*(s')]\right\}, \quad \bar{r}^* = \max_{\pi} \bar{r}(\pi)$
  - $\bar{r}^* + Q^*(s,a) = \mathbb{E}_{(s',r)\sim p(s',r|s,a)}\left[r + \max_{a'} Q^*(s',a')\right]$

TAQ. Is relative return or $V^{\pi}(s)$ finite?

TAQ How to prove relative Bellman eq.?

Algorithm Average Reward SARSA w/ VFA

- initialize w for $q(s,a;w)$
- initialize average reward $\bar{R}$ (e.g. $\bar{R}=0$)
- initialize S
- choose action $A \sim \epsilon-$greedy$(q(S,\cdot;w))$
- repeat for each step
  - take action A, observe R, S'
  - choose action $A' \sim \epsilon-$greedy$(q(S',\cdot;w))$
  - $\delta \leftarrow \left(R + q(S',A';w)\right) - \left(\bar{R} + q(S,A;w)\right)$
  - $w \leftarrow w + \alpha\delta\nabla_w q(S,A;w)$
  - $\bar{R} \leftarrow \bar{R} + \beta\delta$
  - $S \leftarrow S',\ A \leftarrow A'$

$0 = \mathbb{E}\left[r + Q^\pi(s',a')\right] - \left(\bar{r}(\pi) + Q^\pi(s,a)\right)$
$\delta = R + q(S',A';w) - \left(\bar{R} + q(S,A;w)\right) \neq 0$
Assume $\bar{R}$ is correct → update w by min. δ^2
$\nabla_w \delta^2 = -2\delta\nabla_w q(S,A;w)$ (no $\nabla_w q(S',A';w)$. why?)
Assume $q(S,a;w)$ is correct → update $\bar{R}$ by min. δ^2 w.r.t. $\bar{R}$. $\nabla_{\bar{R}}\delta^2 = -2\delta$

TAQ. If $Q^\pi(s,a)$ satisfies Bellman eq., so does $\tilde{Q}^\pi(s,a) = Q^\pi(s,a) + const.$
What is consequence of this?

Algorithm Average Reward Q-Learning w/ VFA (aka R-Learning)

- initialize w for $q(s,a;w)$,
- initialize average reward $\bar{R}$ (e.g. $\bar{R}=0$)
- initialize S
- repeat for each step
  - Choose action $A \sim \epsilon-$greedy$(q(S,\cdot;w))$
  - take action A, observe R, S'
  - $\delta \leftarrow \left(R + \max_{a'} q(S',a';w)\right) - \left(\bar{R} + q(S,A;w)\right)$
  - $w \leftarrow w + \alpha\delta\nabla_w q(S,A;w)$
  - if $\max_a q(S,a;w) == q(S,A;w)$, $\bar{R} \leftarrow \bar{R} + \beta\delta$
  - $S \leftarrow S'$

$0 = \mathbb{E}\left[r + \max_{a'} Q^*(s',a')\right] - \left(\bar{r}^* + Q^*(s,a)\right)$
$\delta = R + \max_{a'} q(S',a';w) - \left(\bar{R} + q(S,a;w)\right) \neq 0$
Assume $\bar{R}$ is correct → update w
Assume $q(S,a;w)$ is correct → update $\bar{R}$
We can use experience replay for sample efficiency.

TAQ. Why update $\bar{R}$ only if $\max_a q(S',a;w) == q(S,A;w)$?

Notable Literatures on Average Reward RL

- A. Schwartz (1993) "A reinforcement learning method for maximizing undiscounted rewards." *ICML* 1993.
  - R-Learning

- S. Mahadevan (1996) "Average reward reinforcement learning: Foundations, algorithms, and empirical results." *Machine learning* 22.1 : 159-195.

- T. Das, A. Gosavi, S. Mahadevan, N. Marchalleck (1999) "Solving semi-Markov decision problems using average reward reinforcement learning." *Management Science* 45.4: 560-574.

- J. Abounadi, D. Bertsekas, V. Borkar (2001) "Learning algorithms for Markov decision processes with average cost." *SIAM Journal on Control and Optimization* 40.3: 681-698.
  - RVI

Time horizon of dynamic system (from Lecture 1)

- Time horizon T (ending time)
  - No-horizon (no known horizon) : T is unknown or infinite
    - indefinite horizon : T finite but unknown
      - positive probability of reaching terminal (=absorbing) state
    - infinite horizon : T is infinite → Continuing task
      - no absorbing (= terminal) state
    - stationarity for no-horizon : stage t does not matter, no notion of age
      - stationary transition and stationary reward : $p(s'|s,a)$, $R(s,a)$
      - stationary decision : $\pi(s), V^{\pi}(s)$
  - Finite horizon : T is **finite** and **known**
    - current time t (= stage, age) matters
    - in state transition and reward at stage t : $p_t(s',r|s,a)$, $R_t(s,a)$
    - decision also depends on the stage t : $\pi_t(s)$, $V_t^{\pi}(s)$
    - at time T, everything gets clear!!
      - $V_T^{\pi}(s)$ (and $\pi_T(s)$) is trivially known.
    - usually, discounting is not used ($\gamma = 1$)

Bellman Eq.

$\tau = (S_0, A_0, R_0, S_1, A_1, R_1, \cdots, S_{T-1}, A_{T-1}, R_{T-1}, S_T)$

Bellman Expectation Eq. ⇒ Policy evaluation

$$V_t^\pi(s) = E\left[R_t + R_{t+1} + \cdots + R_{T-1} \,\middle|\, S_t = s, A_t = \pi_t(s)\right]$$

$$= E\left[R_t + V_{t+1}^\pi(S_{t+1}) \,\middle|\, S_t = s, A_t = \pi_t(s)\right]$$

Bellman Optimality Eq.

$$V_t^*(s) = \boxed{\quad} E\left[R_t + V_{t+1}^*(S_{t+1}) \,\middle|\, S_t = s, A_t = a\right]$$

$$\pi_t^*(s) = \boxed{\quad} E[\quad " \quad | \quad " \quad]$$

DP algorithm (backward DP)

cf. no-horizon ⇒ loopy DP ($V = f(V)$: fixed point iter.)

$V_T^*(s)$: Known

$$V_{T-1}^*(s) = \max_a E\left[R_{T-1} + V_T^*(S_T) \,\middle|\, S_{T-1} = s, A_{T-1} = a\right]$$

$$\vdots \qquad \vdots$$

$$V_0^*(s) = \max_a E\left[R_0 + V_1^*(S_1) \,\middle|\, S_0 = s, A_0 = a\right]$$

Bellman Optimality Eq. w/ Q factor

Let

$$Q_t^*(s,a) = E\Big[\quad \Big| S_t = s, A_t = a\Big]$$

$$V_t^*(s) = \max_a \quad$$

$$Q_t^*(s,a) = E\Big[R_t + \quad \Big| S_t = s, A_t = a\Big]$$

$$\pi_t^*(s) = \arg\max_a \quad$$

$Q_t^*(s,a)$: action value function (cf. $V_t^*(s)$: state value fn.)

Q-factor

└ expected optimal return when taking action $A_t = a$ from state $S_t = s$

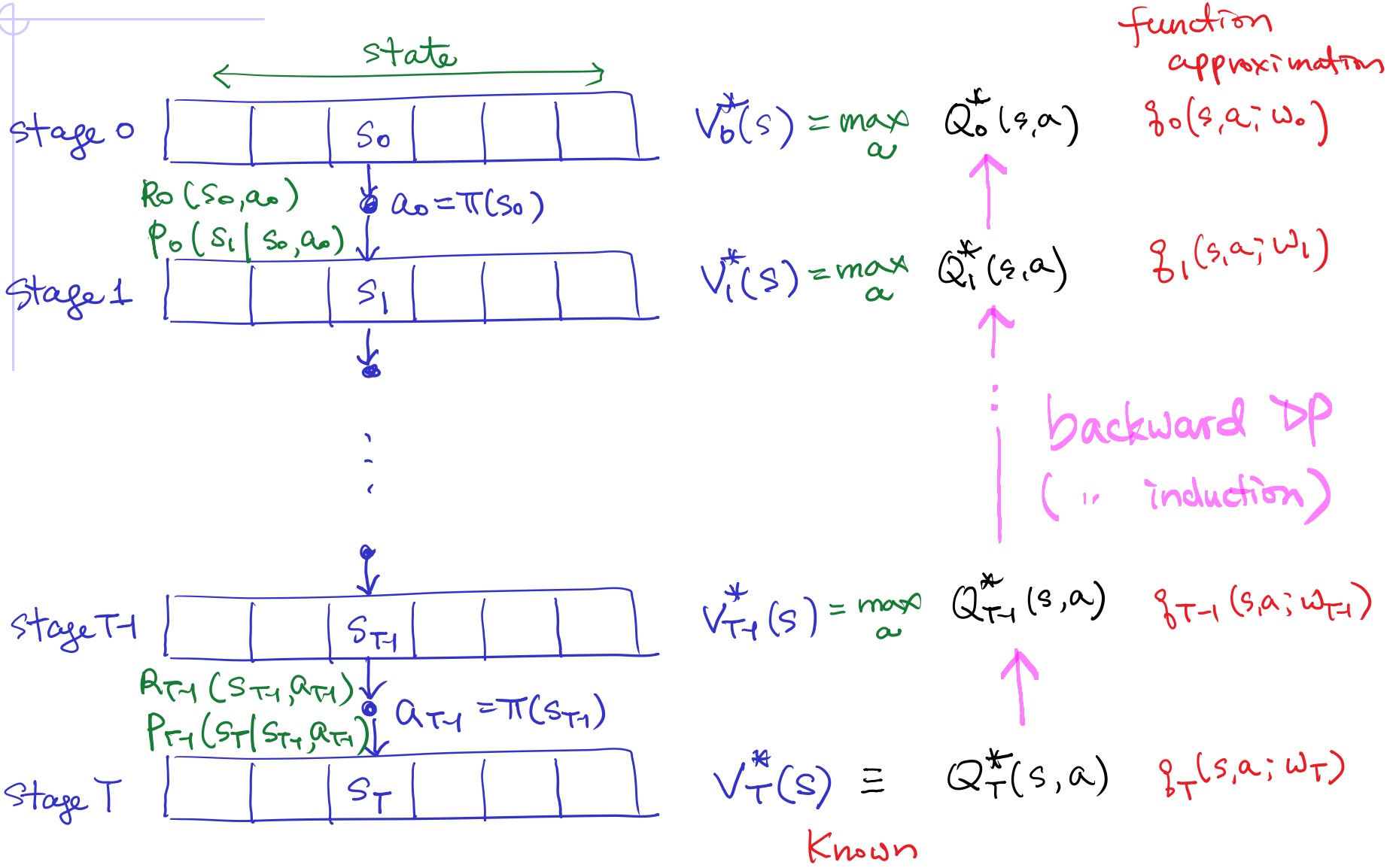
DP algorithm (backward DP) — using Q function

$Q_T^*(s,a)$: Known

$$Q_{T-1}^*(s,a) = E\Big[R_{T-1} + \max_b Q_T^*(S_T, b) \Big| S_{T-1} = s, A_{T-1} = a\Big]$$

$$\vdots \qquad\qquad \vdots$$

$$Q_0^*(s,a) = E\Big[R_0 + \max_b Q_1^*(S_1, b) \Big| S_0 = s, A_0 = a\Big]$$


state
Stage 0
s_0
$R_0(s_0,a_0)$
$a_0=\pi(s_0)$
$P_0(s_1|s_0,a_0)$
Stage 1
s_1
Stage T-1
s_{T-1}
$R_{T-1}(s_{T-1},a_{T-1})$
$a_{T-1}=\pi(s_{T-1})$
$P_{T-1}(s_T|s_{T-1},a_{T-1})$
Stage T
s_T
$V_0^*(s)=\max_a Q_0^*(s,a)$
$V_1^*(s)=\max_a Q_1^*(s,a)$
$V_{T-1}^*(s)=\max_a Q_{T-1}^*(s,a)$
$V_T^*(s) \equiv Q_T^*(s,a)$
Known
function approximation
$f_0(s,a;w_0)$
$f_1(s,a;w_1)$
$f_{T-1}(s,a;w_{T-1})$
$f_T(s,a;w_T)$
backward DP
(= induction)

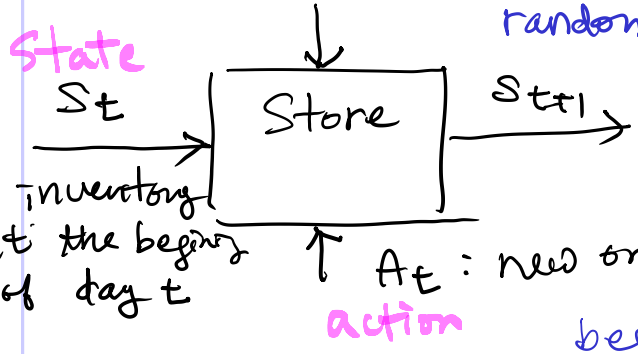
FH example : Inventory Problem

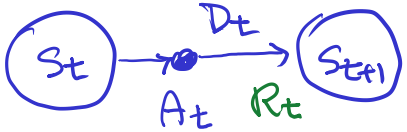
$(x)^+ = \max\{0, x\}$

$(x)^- = \min\{0, x\}$

D_t : demand (during the day)

random demand





→ immediate delivery

- **State transition**

beginning of day $B_t = S_t + A_t$

next day $S_{t+1} =$ [ ]

→ Excess demand $(D_t - B_t)$ is lost

- **Action** A_t : amount of replenishment order (immediate delivery)

- **Reward**

$Sales_t =$ [ ]

$R_t = r \cdot Sales_t - cA_t - wS_t$

r : Sales price

c : purchase price

x : salvage price

w : overnight storage cost

[ ]

- **Time horizon** T : store runs until day T-1

on day T, S_T (leftover) → salvage

$V_T(S_T) =$ [ ] [ ]

FH example: Inventory Problem (T=1)

Inventory problem when T = 1 (single period)

assume $w = 0$

$x < c < r$

$\Rightarrow$ [ ] problem

- Assume $s_0 = 0$ (no initial inventory) $\Rightarrow$ only one state

  Let $F(d) = P[D_0 \le d]$ (cdf)

- optimal policy? (action A: amount of order)

① mean demand matching

$A_0 =$ [ ] $\Rightarrow$ is this optimal? [ ]

② balancing underage and overage cost

overage price $= c - x$, $P[\text{overage}] = P[D_0 \le A] = F(A)$

underage price $= r - c$, $P[\text{underage}] = P[D_0 > A] = 1 - F(A)$

$\Rightarrow (c-x)F(A) = (r-c)(1-F(A))$

$(r-x)F(A) = r-c$

$\Rightarrow F(A) = \frac{r-c}{r-x}$ "critical ratio"

$\Rightarrow A =$ [ ]

Finite Horizon example : Inventory Problem

- Bellman optimality eq.

$$V_t^*(s) = \max_a E\left[R_t + V_{t+1}^*(S_{t+1}) \,\middle|\, S_t, A_t \right]$$

$$B_t = S_t + A_t$$

$$\text{Sales}_t = \min\{B_t, D_t\}$$

$$S_{t+1} = \square$$

$$R_t = r \cdot \text{Sales}_t \quad \square$$

Demand Dist.

$D_t \sim \text{Unif}(1, M)$

$P[D_t = d] = P_d = \square$

$$E_{D_t}[\text{Sales}_t \mid S_t, A_t] = \sum_{d=1}^{M} \square$$

$$E_{D_t}[R_t \mid S_t, A_t] = r \cdot E[\text{Sales}_t] \quad \square$$

$$E_{D_t}[V_{t+1}(S_{t+1}) \mid S_t, A_t] = \sum_{d=1}^{M} \square \cdot P_d$$

$$V_T^*(S_T) = \square$$

Inventory.py

```
import numpy as np
T = 10
M = 10
M1 = M + 1
x = 2
c = 10.5
r = 12
w = 0.3

def p(d):
    return 1/M
```

```
def solveInventoryProblem():
    V = np.zeros((T+1,M1))
    for s in range(M1):
        V[T][s] = x * s - w * s
    pi = np.zeros((T,M1), dtype='int')
    Q = np.zeros(M1)

    for tp1 in range(T,0,-1):
        t = tp1 - 1
        for s in range(M1):
            for a in range(M1-s):
                b = s + a
                Esales = 0
                Q[a] = 0
                for d in range(1,M1):
                    Esales += min(b, d) * p(d)
                    remain = max(0, b-d)
                    Q[a] += V[t+1][remain] * p(d)
                Ereward = r*Esales - c*a - w*s
                Q[a] += Ereward
            a = np.argmax(Q)
            pi[t][s] = a
            V[t][s] = Q[a]

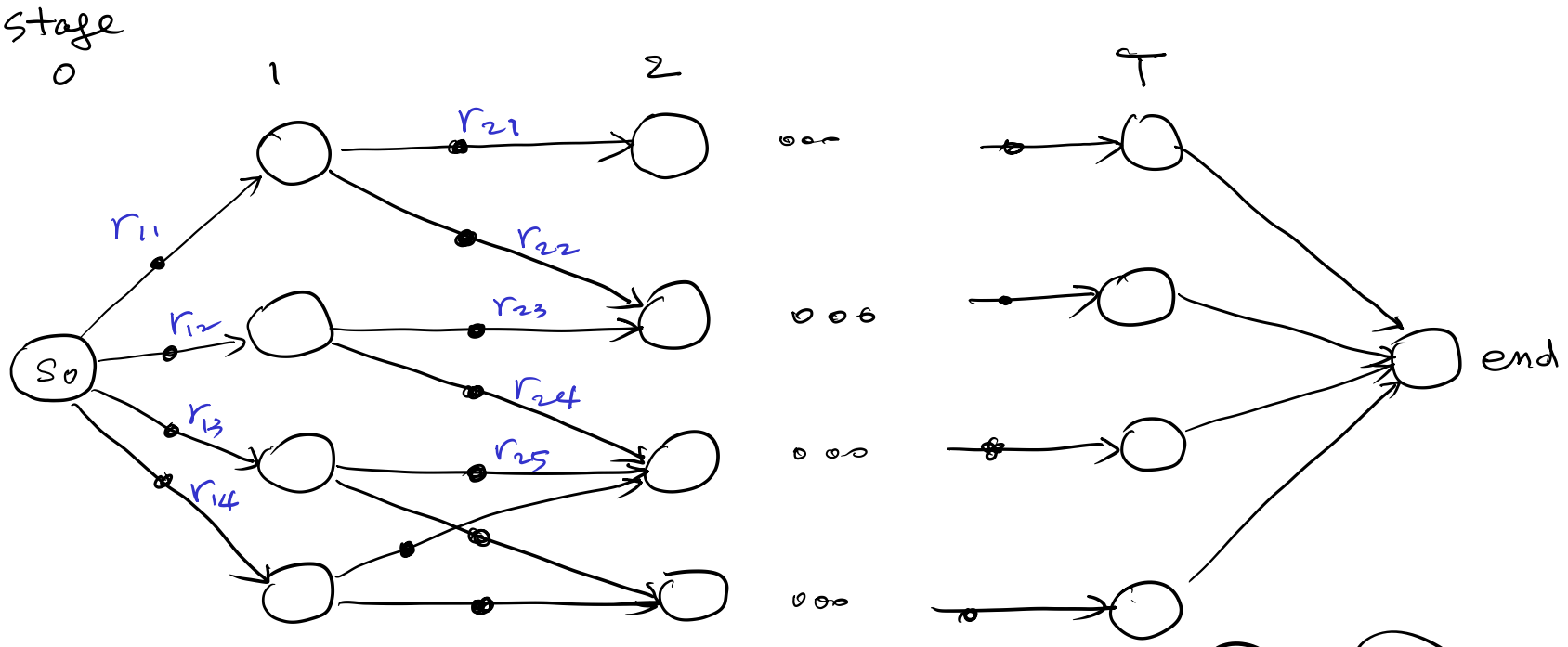
    print(pi)
```

Policy (# of order) at t

$\Pi_0(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_1(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_2(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_3(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_4(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_5(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_6(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_7(s)$: [8 7 6 5 4 3 2 1 0 0 0]
$\Pi_8(s)$: [6 5 4 3 2 1 0 0 0 0 0]
$\Pi_9(s)$: [2 1 0 0 0 0 0 0 0 0 0]

$\pi_7(s_7=6) = 2$: at $t=7$, if $s_7=6$, order $A_7=2$

A1. Deterministic Fin. Hor. Problem ⇔ Shortest path prob.



- Goal : to fine the "maximum return path" from s_0 to end
- In general, longest path is harder to find than shortest path

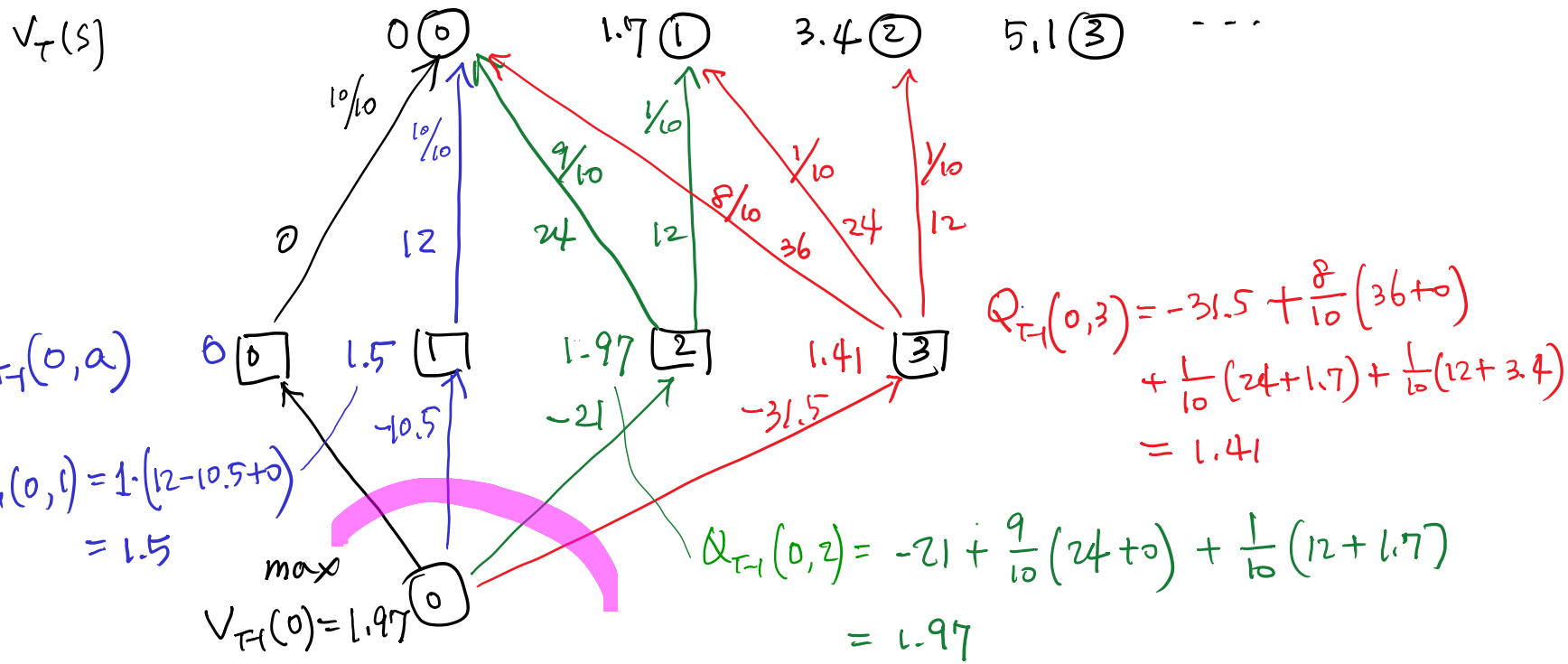
  This case, every path has the same length

  ⇒ can be solved easily.

  Change r_{ij} ⇒ $M - r_{ij}$ Apply Dijkstra

A2. Inventory problem : flow of computation

$V_T(s) = x \cdot s - w \cdot s = (x-w)s = (2-0.3)s = 1.7s$



A3. Inventory problem: interpreting policy

Policy (# of order) at t

$\Pi_0(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_1(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_2(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_3(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_4(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_5(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_6(s)$: [9 8 7 6 5 4 3 2 1 0 0]
$\Pi_7(s)$: [8 7 6 5 4 3 2 1 0 0 0]
$\Pi_8(s)$: [6 5 4 3 2 1 0 0 0 0 0]
$\Pi_9(s)$: [2 1 0 0 0 0 0 0 0 0 0]

- $t = T-1 = 9$, $s = 0$: $\pi_9(0) = 2$. why not 3?
  - → afraid of leftover
  - if we order 3rd item (#3)

#3 not sold:
- $P[\text{overage}] = P[D_9 \le 2] = 2/10$
- overage cost $= c - (x - w) = 10.5 - (2 - 0.3) = 8.8$

#3 sold:
- $P[\#3 : \text{sold}] = P[D_9 > 2] = 8/10$
- revenue by #3 $= r - c = 12 - 10.5 = 1.5$

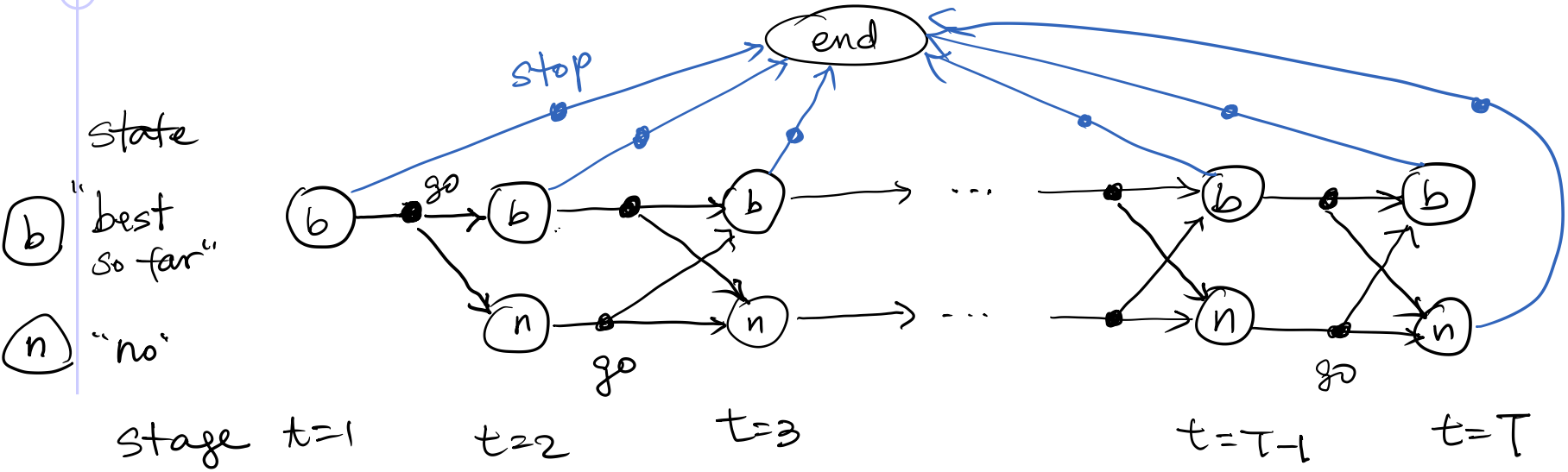
We order #3 if $E[\text{overage cost}] < E[\text{revenue}]$ ⇒ ordering #3 is not profitable

$8.8 \cdot 2/10 = 1.76$ vs 1.2

- at $t = 0$, $s = 0$: $\pi_0(0) = 9$, why not 10? ⇒ storage cost matters
  - if we order 10th item (#10)
  - $P[\text{overage}] = P[D_0 \le 9] = 9/10$, overage cost $= 0.3$ ⇒ $E[\text{overage cost}] = 0.27$
  - $P[\#10 : \text{sold}] = P[D_0 > 9] = 1/10$, revenue by #10 $= 1.5$ ⇒ $E[\text{revenue}] = 0.15$
  - Ordering #10 is not profitable

A4.Finite Horizon example : Secretary Problem

- Optimal stopping problem : Go or Stop?
  - Cat't go back in time
  - Parking, Blackjack card game, dating, …
- Secretary problem : a typical optimal stopping problem
  - Hiring a secretary
  - T candidates (applicants)
  - After interviewing a candidate c_t,
    - we can compare c_t against predecessors (c_1, …, c_{t-1})
    - i.e. we can judge if c_t == best(c_1 .. c_t)
    - then, we should whether to hire or reject c_t
    - no assumptions on the distribution of the quality of the candidates
  - PCS (Prob. of correct selection) : P[c_t ==best(c_1 .. c_T)] if we hire c_t
  - Optimal policy to maximize PCS ?



state $S_t = \begin{cases} b & \text{if } C_t = \text{best}(C_1 \cdots C_t) \\ n & \text{" } C_t \neq \text{" } \\ \text{end} & \text{if } A_t = \text{stop or } t = T+1 \end{cases}$ (∵ we can remember only the past)

action $A_t = \begin{cases} \text{stop} : \text{stop here and hire } C_t \\ \text{go} : \text{go on to } C_{t+1} \end{cases}$

- Immediate Reward $R_{t+1}(S_t, A_t)$



prob. of containing the best: t/T $\quad 1-t/T$

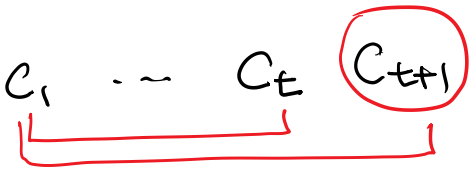
S_t \ A_t	stop	go
b	t/T	0
	0	0

$$R = \begin{cases} 1 & \text{for CS} \\ 0 & \text{o.w} \end{cases}$$

$$\Downarrow$$

$$E[R] = PCS$$

- State transition

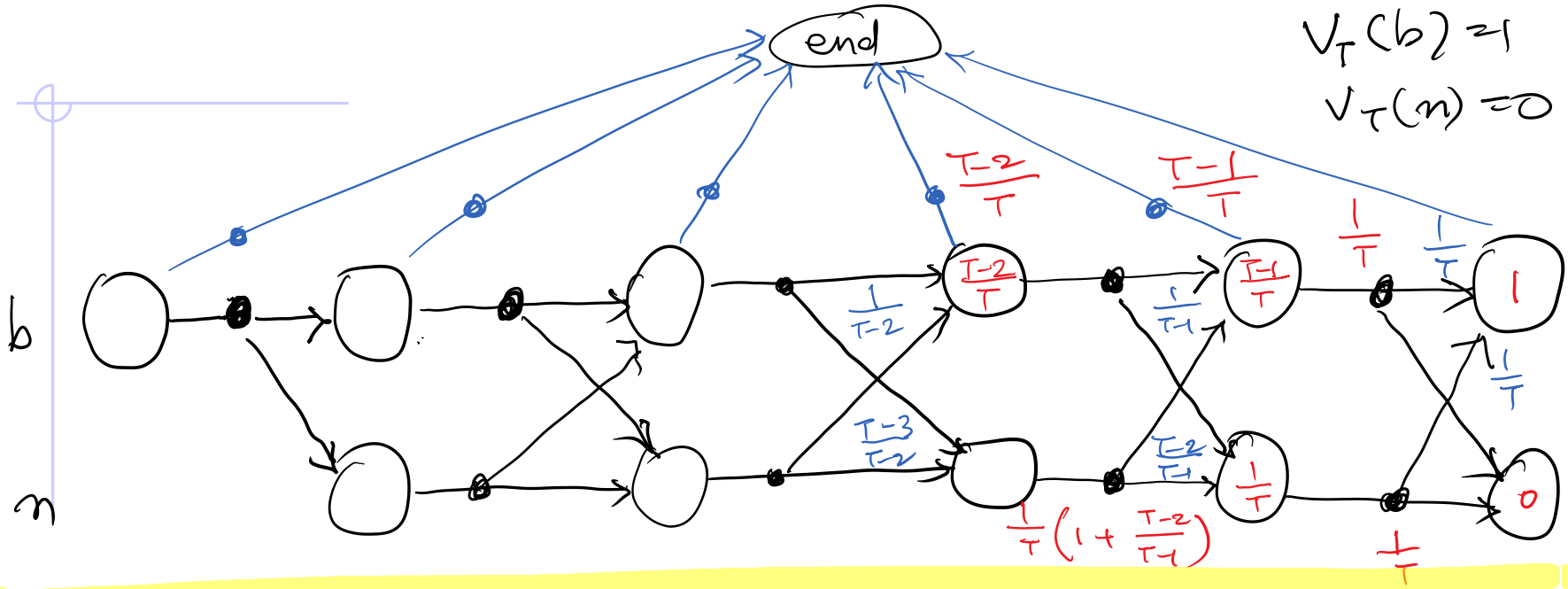


$$P_t(S_{t+1} = end \mid S_t, A_t = stop) = 1$$

$$P_t(S_{t+1} = b \mid S_t, A_t = go)$$

$$= P(c_{t+1} = best(c_1 \cdots c_{t+1})) = 1/(t+1)$$

$$P_t(S_{t+1} = n \mid S_t, A_t = go) = 1 - 1/(t+1) = \frac{t}{t+1}$$



$$V_t(s) = \max_a E\left[R_{t+1} + V_{t+1}(S_{t+1}) \mid S_t = s,\ A_t = a\right]$$

$$V_t(b) = \max \begin{cases} \text{stop} & t/T \\ \text{go} & 0 + \underbrace{P_t(b \mid b, \ )}_{1/(t+1)} V_{t+1}(b) + \underbrace{P_t(n \mid b, \text{go})}_{t/(t+1)} V_{t+1}(n) \end{cases}$$

$$V_t(n) = \max \begin{cases} \text{stop} & 0 \\ & 0 + \frac{1}{t+1} V_{t+1}(b) + \frac{t}{t+1} V_{t+1}(n) \end{cases}$$

Secretary.py

```
import numpy as np

# state b : 0, n : 1
b = 0
n = 1

def solveSecretaryProblem (T):
    V = np.zeros((T+1,2)) # value (PCS)
    V[T][b] = 1
    V[T][n] = 0
    pi = np.zeros(T+1, dtype='int' ) # policy

    for tp1 in range(T, 0, -1):
        t = tp1 - 1
        go = 0.0 + 1/tp1*V[t+1][b] + t/tp1*V[t+1][n]
        stop = t / T
        if (go > stop):  # V[t][b] = max(go, stop)
            V[t][b] = go
            pi[t] = 1
        else:
            V[t][b] = stop
            pi[t] = 0
        V[t][n] = max(0, go)

    print (V)
    print (pi)
```

(T=20) $V_t[b]$	$V_t[n]$	$\pi_t(b)$
[0.38420888	0.38420888]	go
[0.38420888	0.38420888]	go
[0.38420888	0.38420888]	go
[0.38420888	0.38420888]	go
[0.38420888	0.38420888]	go
[0.38420888	0.38420888]	go
[0.38420888	0.38420888]	go
[0.38420888	0.38420888]	go
[0.4	0.38195301]	stop
[0.45	0.37344713]	stop
[0.5	0.3593857]	stop
[0.55	0.34032427]	stop
[0.6	0.31671739]	stop
[0.65	0.28894384]	stop
[0.7	0.25732413]	stop
[0.75	0.222133]	stop
[0.8	0.18360853]	stop
[0.85	0.14195906]	stop
[0.9	0.09736842]	stop
[0.95	0.05]	stop
[1.	0.]]	stop

40%



36.8%
as $T \rightarrow \infty$

Pass-m policy : Pass first m candidates, (A_t=go if $t<m$)
then hire c_t if $c_t == best(c_1 .. c_t)$ (A_t=stop if $t \geq m$ & S_t=b)

π_t

S_t	$t<m$	$t \geq m$
b	go	stop
n	go	go

$P_m = V_1^{\pi}(b)$ when π = pass-m policy

